

1 Exclusión mutua (control de acceso a recursos compartidos)

Definición:

La **exclusión mutua** es un mecanismo que **asegura que solo un hilo o proceso pueda acceder a un recurso compartido a la vez**.

👉 Es fundamental en **programación concurrente** para evitar **condiciones de carrera (race conditions)**, donde varios hilos intentan modificar el mismo recurso al mismo tiempo.

Ejemplo:

Supongamos que dos hilos acceden a una variable compartida **saldo** en una cuenta bancaria.

Sin exclusión mutua:

```
// Ejemplo de condición de carrera
saldo = saldo - 100; // Hilo 1
saldo = saldo - 100; // Hilo 2
```

Si ambos leen el mismo valor inicial, podrían dejar el saldo incorrecto.

Con exclusión mutua:

```
synchronized (this) { // Bloque sincronizado
    saldo = saldo - 100;
}
```

✅ Solo un hilo puede ejecutar ese bloque a la vez.

Formas de implementar exclusión mutua en Java

Mecanismo	Descripción
<code>synchronized</code>	Bloquea el acceso a un método o bloque de código.

<code>Lock</code> (de <code>java.util.concurrent.locks</code>)	Ofrece más control (bloqueo, desbloqueo manual, tiempo de espera, etc.).
<code>ReentrantLock</code>	Permite que el mismo hilo adquiera el bloqueo varias veces.
<code>Semaphore</code>	Limita el número de hilos que pueden acceder simultáneamente.
<code>Atomic</code> (como <code>AtomicInteger</code>)	Proporciona operaciones atómicas sin bloqueos explícitos.

2 Interbloqueo (Deadlock)

Definición:

Un **interbloqueo** ocurre cuando **dos o más hilos esperan indefinidamente por recursos que otros hilos ya poseen**, bloqueándose entre sí.

👉 Es como cuando dos coches se cruzan en una calle estrecha y ninguno puede avanzar porque esperan que el otro retroceda.

Ejemplo clásico de interbloqueo en Java

```
public class DeadlockDemo {
    private final Object recursoA = new Object();
    private final Object recursoB = new Object();

    public static void main(String[] args) {
        DeadlockDemo ejemplo = new DeadlockDemo();
        ejemplo.iniciar();
    }

    public void iniciar() {
        Thread t1 = new Thread(() -> {
            synchronized (recursoA) {
                System.out.println("Hilo 1 bloqueó recurso A");
                try { Thread.sleep(100); } catch
(InterruptedExcepcion e) {}
                synchronized (recursoB) {
```

```

        System.out.println("Hilo 1 bloqueó recurso B");
    }
}

});

Thread t2 = new Thread(() -> {
    synchronized (recursoB) {
        System.out.println("Hilo 2 bloqueó recurso B");
        try { Thread.sleep(100); } catch
(InterruptedExcepción e) {}
        synchronized (recursoA) {
            System.out.println("Hilo 2 bloqueó recurso A");
        }
    }
});

t1.start();
t2.start();
}
}

```

✏️ Aquí, el **hilo 1** bloquea **recursoA** y espera **recursoB**, mientras que el **hilo 2** bloquea **recursoB** y espera **recursoA**.
→ Ninguno puede continuar: **deadlock**.

⚙️ Cómo reproducir un deadlock

1. Crear dos (o más) hilos.
 2. Hacer que cada uno bloquee un recurso diferente.
 3. Hacer que luego intente bloquear el recurso del otro.
 4. Añadir un pequeño `Thread.sleep()` entre bloqueos para aumentar la probabilidad del bloqueo cruzado.
-

🧩 Cómo detectar un deadlock

1 En ejecución (manual):

En IntelliJ o en consola puedes usar:

jconsole

o

jvisualvm

- → Herramientas de monitoreo de la JVM que muestran los hilos bloqueados y recursos en espera.

2 Programáticamente:

Usando el **ThreadMXBean** del paquete `java.lang.management`:

```
import java.lang.management.*;

public class DetectarDeadlock {
    public static void main(String[] args) throws
    InterruptedException {
        ThreadMXBean bean = ManagementFactory.getThreadMXBean();
        long[] ids = bean.findDeadlockedThreads();

        if (ids != null) {
            System.out.println("⚠ Deadlock detectado entre
hilos:");
            ThreadInfo[] infos = bean.getThreadInfo(ids);
            for (ThreadInfo info : infos) {
                System.out.println(info.getThreadName());
            }
        } else {
            System.out.println("✅ Sin deadlocks detectados.");
        }
    }
}
```

🧠 Cómo evitar deadlocks

Estrategia	Descripción
Orden consistente de bloqueo	Asegurarte de que todos los hilos adquieran los recursos en el mismo orden.

Bloqueos con tiempo límite	Usar <code>tryLock(timeout)</code> en lugar de <code>lock()</code> para evitar esperas infinitas.
Evitar bloqueos innecesarios	Minimizar las secciones sincronizadas.
Diseño sin dependencias cíclicas	Evitar que los recursos dependan unos de otros en cadena.

Ejemplo de solución con orden de bloqueo

```
// Ambos hilos adquieren los bloqueos en el mismo orden
synchronized (recursoA) {
    synchronized (recursoB) {
        // Acceso seguro sin interbloqueo
    }
}
```

Resumen final

Concepto	Definición	Prevención
Exclusión mutua	Controla el acceso a recursos compartidos para evitar conflictos.	<code>synchronized</code> , <code>Lock</code> , <code>Semaphore</code> , etc.
Interbloqueo (deadlock)	Estado en que dos o más hilos esperan indefinidamente por recursos que el otro posee.	Orden de bloqueo, <code>tryLock()</code> , diseño sin dependencias cíclicas.